GnoSwap Architecture

GnoSwap Architecture Document

> Upgradeable Concentrated Liquidity AMM with Runtime Version Management

---

Version: 1.0

Table of Contents

1. Overview
2. Core Contracts
3. Layer Architecture
4. Feature Flows
5. Reward Distribution

---

1. Overview

1.1 What is GnoSwap?

GnoSwap is a **Concentrated Liquidity AMM** built on the Gno blockchain. Based on the Uniswap V3 concentrated liquidity model, it provides an upgradeable architecture that leverages the characteristics of the Gno chain.

Key Features:

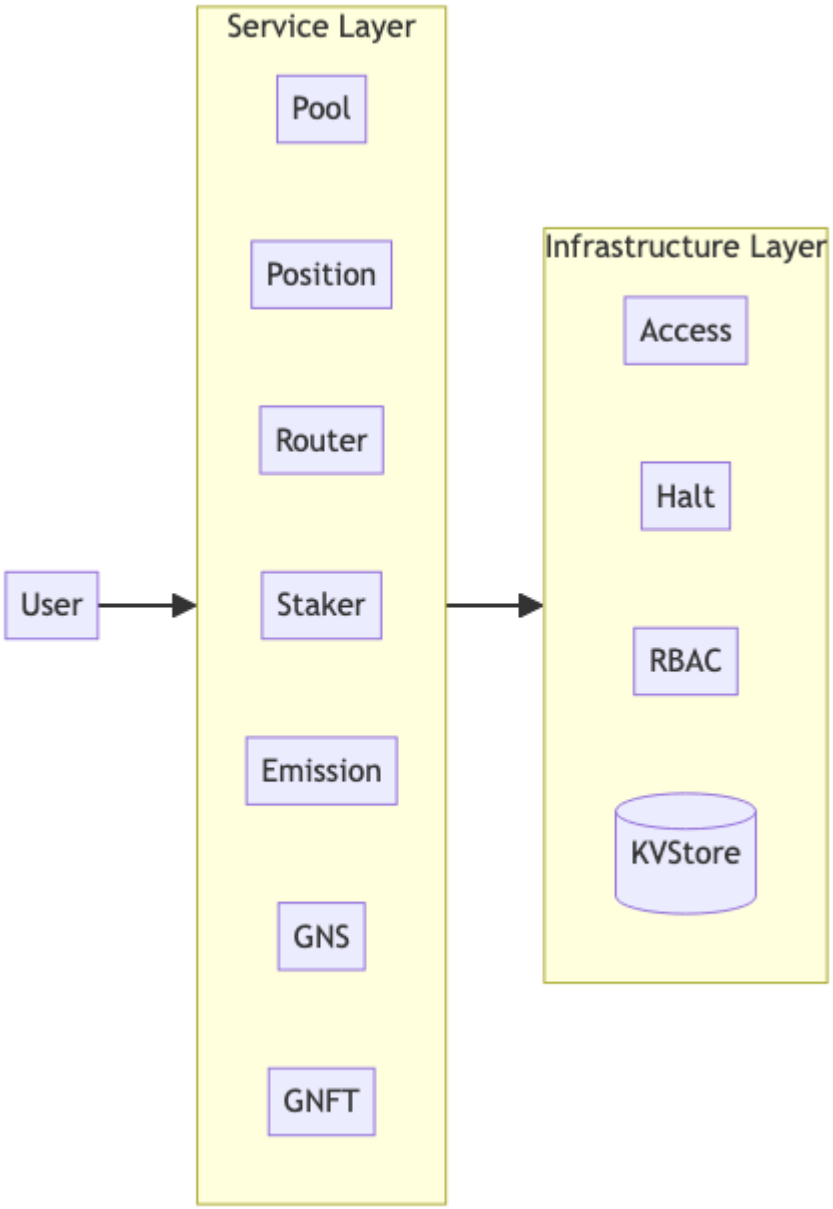
- **Concentrated Liquidity**: Liquidity providers (LPs) can concentrate their capital within specific price ranges to maximize capital efficiency. Up to 4000x capital efficiency can be achieved compared to traditional AMMs.

- **Upgradeable Architecture**: Through the Version Manager pattern, contract logic can be upgraded without service interruption. Instant transition to new versions without data migration.

- **Integrated Reward System**: Unified management of GNS token emission and external incentives. LPs can earn additional rewards beyond swap fees.
- **NFT-based Positions**: Each liquidity position is represented as a unique NFT, enabling individual management, transfer, and collateral utilization.

1.2 System Architecture

GnoSwap consists of a 2-layer architecture. User requests are processed through Service Layer contracts, utilizing Infrastructure Layer base services.



Architecture Description:

- **Service Layer**: Domain contracts that process business logic. They handle user requests and provide core functionality for each domain.

- **Infrastructure Layer**: Provides base services. Responsible for common functions used by all Services such as data storage, permission management, and emergency halt.

1.3 Directory Structure

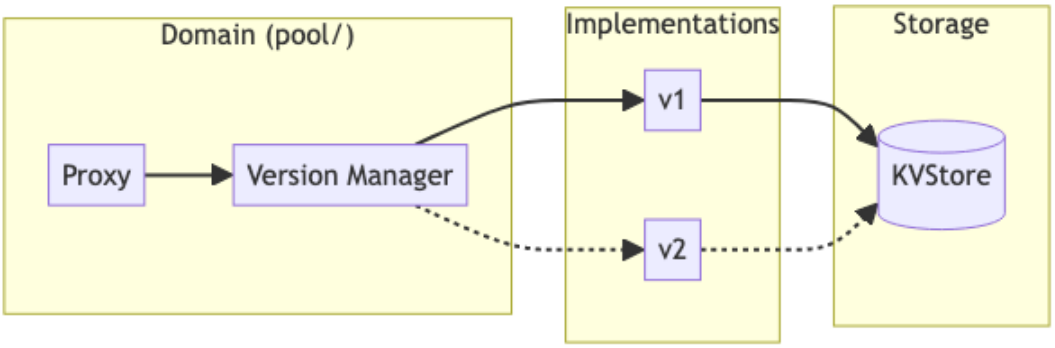
```
contract/
├── p/gnoswap/                  # Packages (stateless)
│   ├── int256/, uint256/       # Big number operations
│   ├── gnsmath/                # Swap math operations
│   ├── store/                  # KVStore abstraction
│   └── version_manager/        # Version management system
│
└── r/gnoswap/                  # Realms (stateful)
    ├── pool/                   # AMM pools
    │   ├── pool.gno            # Proxy interface
    │   └── v1/                 # Implementation
    ├── position/               # Position management
    ├── router/                 # Swap routing
    ├── staker/                 # Reward system
    ├── emission/               # Token emission
    ├── gns/                    # GNS token
    ├── gnft/                   # Position NFT
    ├── access/                 # Permission management
    └── halt/                   # Emergency halt
```

Packages vs Realms:

- **Packages (p/)**: Pure libraries without state. Provide math operations, utility functions, etc.
- **Realms (r/)**: Smart contracts with state. Manage actual business logic and data.

1.4 Version Manager Pattern

The core design pattern of GnoSwap is the Version Manager that enables zero-downtime upgrades.



How it Works:

1. **Proxy Layer**: Provides a stable public interface. The API seen from outside remains unchanged.

2. **Version Manager**: Tracks the currently active implementation and manages version transitions.

3. **Implementation Layer**: Version-specific packages where actual business logic is implemented. Multiple versions (v1, v2, etc.) can coexist.

4. **Storage Layer**: A single KVStore shared by all versions. No data migration required during version transitions.

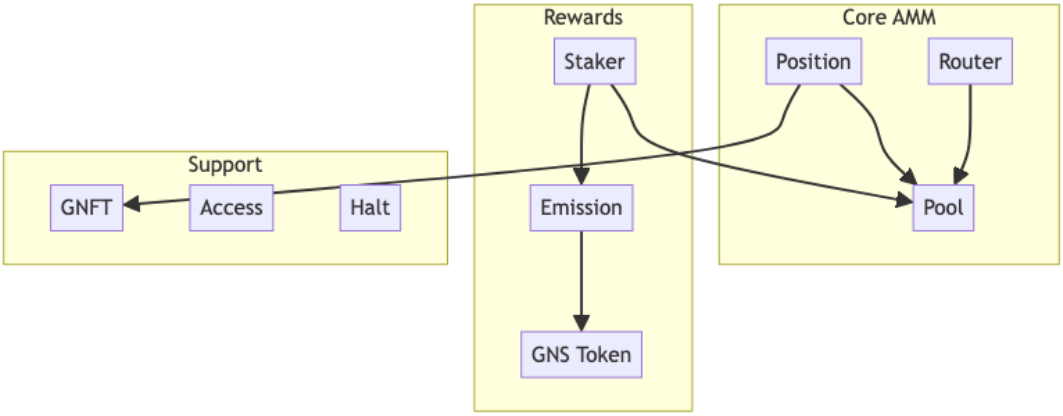
Upgrade Process:

```
1. Deploy and register new version (v2)
2. Admin calls ChangeImplementation("pool/v2")
3. Version Manager changes current implementation pointer to v2
4. v2 logic is immediately active (zero-downtime)
5. v1 remains read-only (rollback possible)
```

2. Core Contracts

2.1 Contract Overview

GnoSwap consists of contracts that are clearly separated by their roles.



Contract	Role	Upgradeable
Pool	Concentrated liquidity AMM core logic	Yes
Position	LP position NFT management	Yes
Router	Multi-hop swap routing	Yes
Staker	Liquidity mining rewards	Yes
Emission	GNS emission schedule	No

Contract	Role	Upgradeable
GNS	Governance token (GRC20)	No
GNFT	Position NFT (GRC721)	No
Access	Role-based permission management	No
Halt	Emergency halt system	No

2.2 Pool Contract

The Pool contract is the core of GnoSwap, handling all logic for the concentrated liquidity AMM.

Responsibilities:

- Pool creation and initialization
- Tick-based liquidity management
- Swap execution and price calculation
- Fee accumulation and distribution
- TWAP oracle data management

Key Functions:

Function	Description	Caller
CreatePool	Create new trading pair pool	User
Mint	Add liquidity	Position
Burn	Remove liquidity	Position
Swap	Token exchange	Router
Collect	Collect fees/principal	Position

Pool State Data:

```
Pool State
├── sqrtPriceX96      Current price (Q64.96 format)
├── tick              Current tick index
├── liquidity         Active liquidity in current price range
├── feeGrowthGlobal   Global fee accumulator
├── ticks             Per-tick liquidity info (AVL Tree)
├── positions         Per-position state (AVL Tree)
└── observations      TWAP oracle data
```

Fee Tiers:

Pools are created with one of 4 fee tiers. Higher fees are suitable for volatile pairs.

Fee	Tick Spacing	Use Case
0.01% (100)	1	Stablecoin pairs
0.05% (500)	10	Stable pairs

Fee	Tick Spacing	Use Case
0.3% (3000)	60	General pairs
1% (10000)	200	Volatile pairs

2.3 Position Contract

The Position contract manages user liquidity positions as NFTs.

Responsibilities:

- Liquidity position creation/modification/deletion
- Position NFT minting and metadata management
- Accumulated fee collection processing
- Position repositioning (price range change)

Key Functions:

Function	Description
Mint	Create new position, mint NFT
IncreaseLiquidity	Add liquidity to existing position
DecreaseLiquidity	Remove liquidity from position
CollectFee	Collect accumulated fees
Reposition	Change price range

Position Data:

Each position has a unique NFT ID and contains the following information:

```
Position
├── tokenId           NFT ID
├── owner             Owner address
├── token0, token1    Pool token pair
├── fee               Fee tier
├── tickLower         Lower price tick
├── tickUpper         Upper price tick
├── liquidity         Liquidity amount
└── tokensOwed        Uncollected tokens
```

2.4 Router Contract

The Router contract executes swaps through optimal paths.

Responsibilities:

- Swap path parsing and validation
- Multi-hop swap execution
- Slippage protection

- Native GNOT wrapping/unwrapping

Key Functions:

Function	Description
ExactInSwapRoute	Fixed input amount swap
ExactOutSwapRoute	Fixed output amount swap
DrySwapRoute	Swap simulation (no state change)

Route Format:

```
# Single-hop
"tokenIn:tokenOut:fee"
Example: "gno.land/r/demo/bar:gno.land/r/demo/baz:3000"

# Multi-hop
"token1:token2:fee1*POOL*token2:token3:fee2"
Example: "bar:baz:3000*POOL*baz:qux:500"

# Quote (ratio per path)
"30,70"  → First path 30%, second path 70%
```

2.5 Staker Contract

The Staker contract manages liquidity mining rewards.

Responsibilities:

- Position staking/unstaking
- Internal reward (GNS) calculation and distribution
- External incentive management
- Pool tier-based reward distribution

Key Functions:

Function	Description
StakeToken	Stake position NFT
UnStakeToken	Unstake and receive rewards
CollectReward	Collect rewards only (maintain stake)
MintAndStake	Create position + stake in one shot
CreateExternalIncentive	Create external reward pool

Reward Types:

1. **Internal Rewards (GNS)**: GNS issued from Emission is distributed according to pool tier.

2. **External Incentives**: Anyone can set up custom token rewards for specific pools.

Pool Tier System:

Pools are classified into tiers 1-10, with higher tiers receiving more GNS rewards.

2.6 Supporting Contracts

GNS Token:

GnoSwap's governance token. Follows GRC20 standard, only Emission contract can mint.

- Initial mint: 100 trillion GNS
- Maximum supply: 1,000 trillion GNS
- Emission period: 12 years (with halving)

GNFT:

Position NFT contract. Follows GRC721 standard, generates dynamic SVG metadata.

Access:

Role-based permission management system. Manages addresses by role such as admin, governance, pool, position, etc.

Halt:

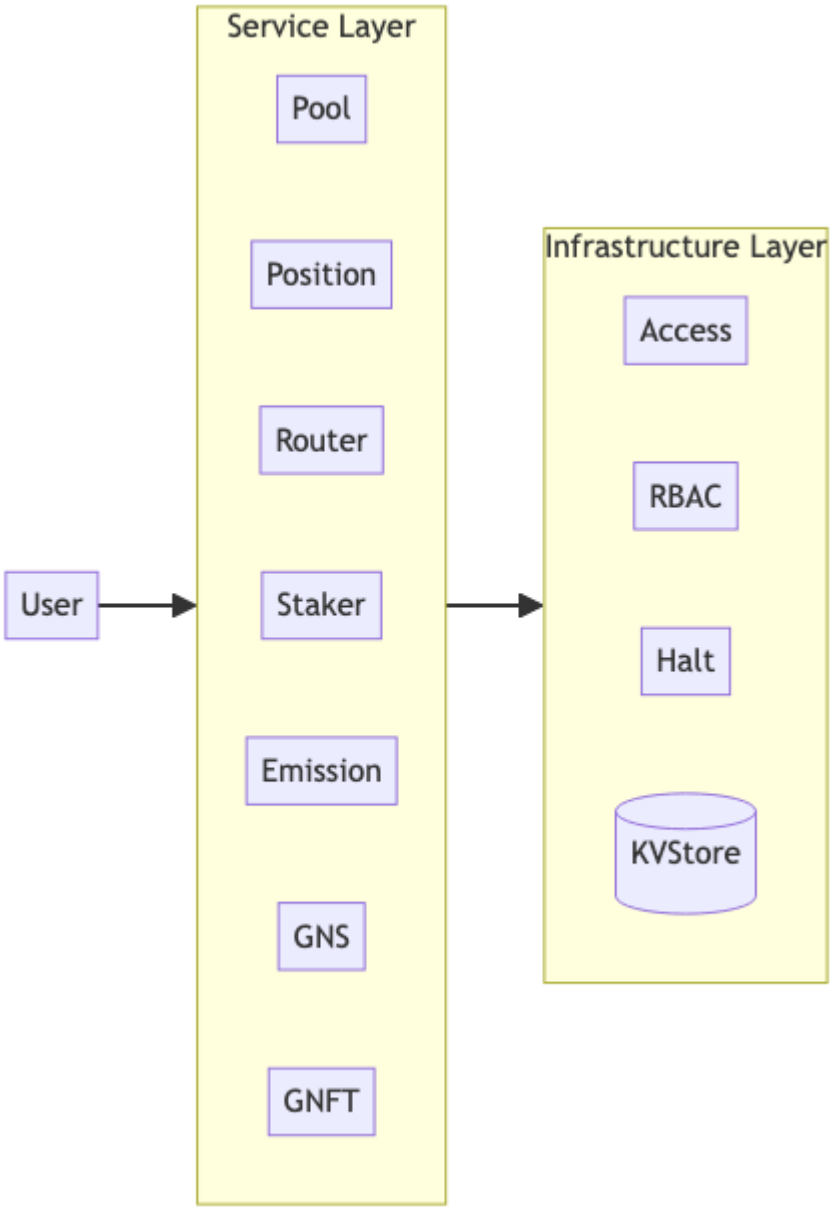
Emergency halt system supporting 4 halt levels:

Level	Description
None	Normal operation
SafeMode	Only core operations allowed
Emergency	Only withdrawals allowed
Complete	All operations halted

3. Layer Architecture

3.1 Two-Layer Structure

GnoSwap uses a 2-layer architecture consisting of Service and Infrastructure layers. Each layer has clear responsibilities, with the Service Layer depending on the Infrastructure Layer.



3.2 Service Layer

The Service Layer consists of domain contracts that process business logic.

Contract	Responsibility
Pool	Concentrated liquidity pool management, swap execution, fee accumulation
Position	LP position CRUD, NFT minting, fee collection
Router	Swap path parsing, multi-hop execution, slippage validation
Staker	Staking management, reward calculation, external incentives
Emission	GNS emission schedule management, distribution ratio calculation

Contract	Responsibility
GNS	GNS token minting/transfer/burning
GNFT	Position NFT minting/burning, metadata management

Characteristics:

- Entry point for user requests
- Domain-specific business logic implementation
- Interaction with other Service contracts
- Utilization of Infrastructure services

3.3 Infrastructure Layer

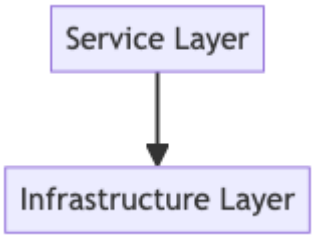
The Infrastructure Layer provides base services commonly used by all Services.

Component	Responsibility
Access	Role-based access control
RBAC	Role address management
Halt	Emergency halt system
KVStore	Persistent data storage

Characteristics:

- Commonly used by all Services
- Provides security and stability foundation
- Independent of domain logic
- Unaware of upper layers (unidirectional dependency)

3.4 Layer Dependencies

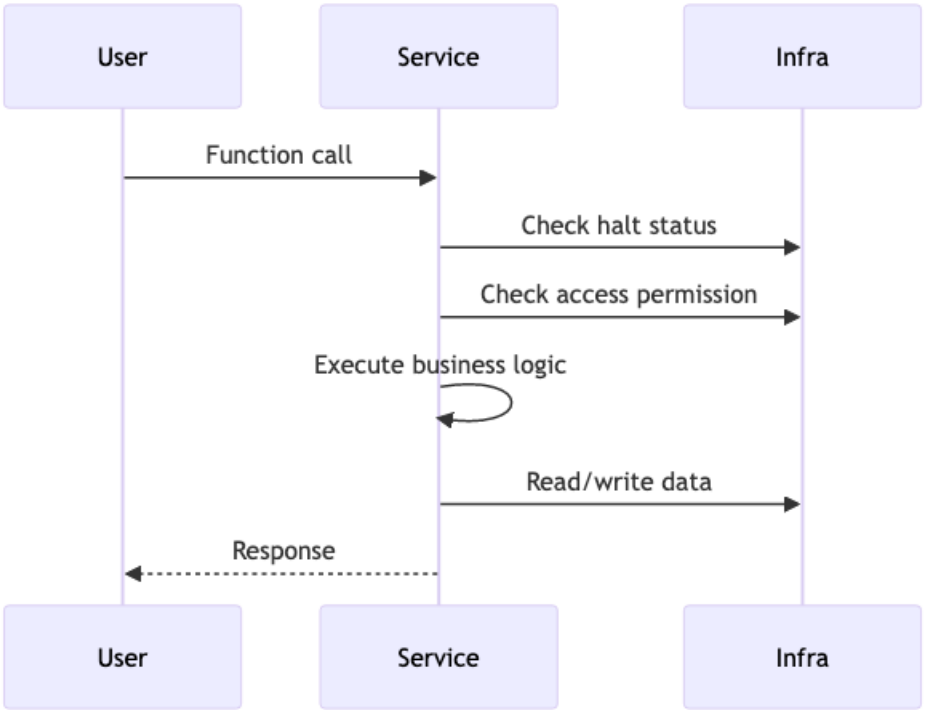


Dependency Rules:

1. Service Layer uses data and services from the Infrastructure Layer.
2. Infrastructure Layer is unaware of the Service Layer.
3. Inter-contract calls within the Service Layer are allowed.

3.5 Data Flow

A typical flow for processing user requests:



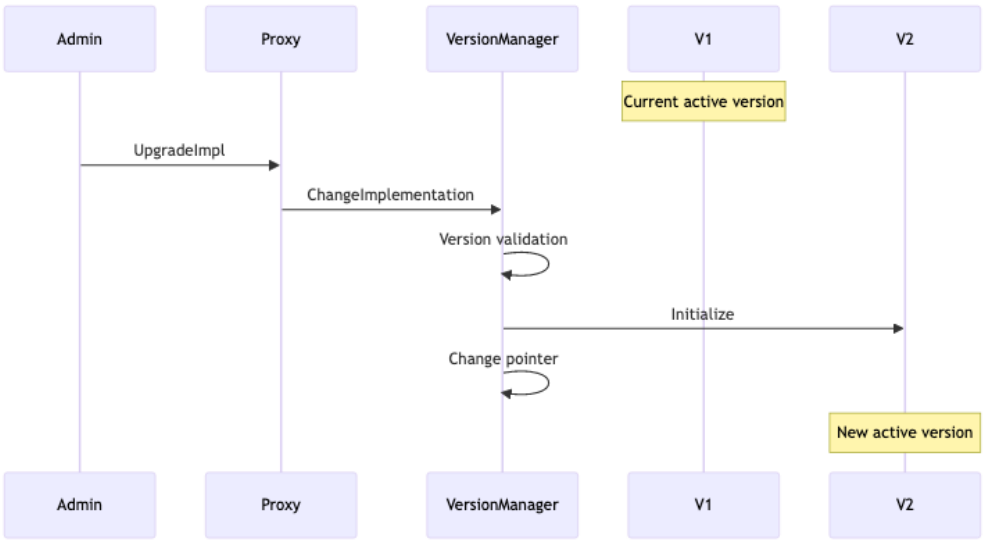
Processing Steps:

1. **Service Layer**: Receive user request, check halt/access, execute business logic
2. **Infrastructure Layer**: Permission verification, data persistence
3. **Response**: Return result to user

4. Feature Flows

4.1 Version Upgrade Flow

Contract upgrades proceed without service interruption.



Upgrade Process:

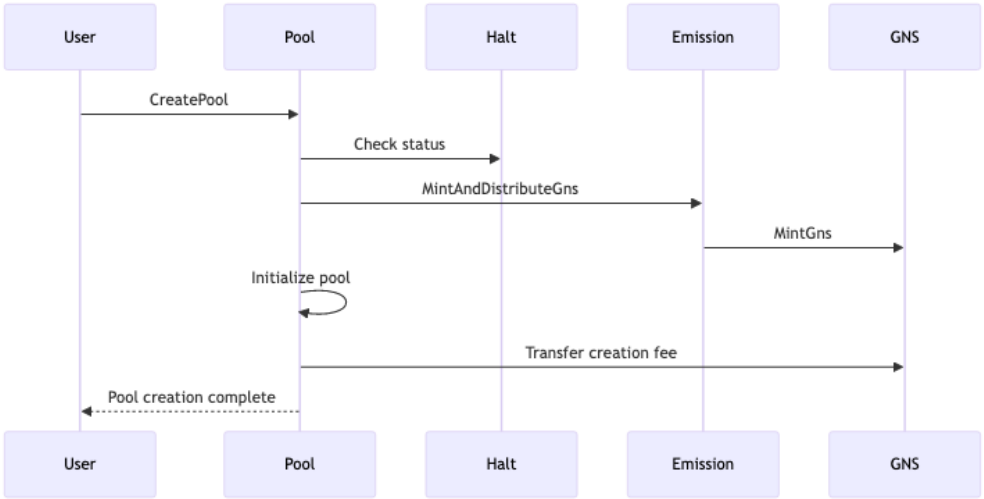
1. Admin calls `UpgradeImpl` with new version package path
2. Version Manager verifies the version is registered
3. Execute new version's initializer (using same KVStore)
4. Change `currentImplementation` pointer to new version
5. All subsequent calls are routed to new version

Characteristics:

- No data migration required (same storage)
- Immediate application (zero-downtime)
- Rollback to previous version possible

4.2 Pool Creation Flow

Process for creating a new trading pair pool.



Creation Process:

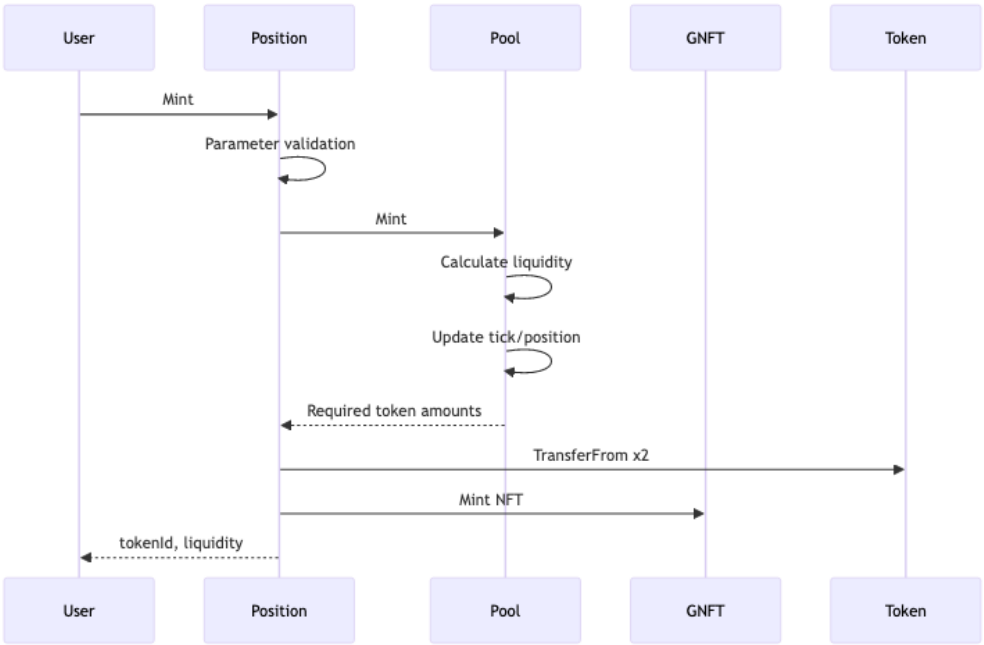
1. User calls `CreatePool` with token0, token1, fee, initial price
2. Check Halt status (fails if halted)
3. Trigger Emission (GNS minting and distribution)
4. Initialize pool structure:
   - Sort token order (token0 < token1)
   - Calculate initial tick from sqrtPriceX96
   - Initialize ticks, positions AVL Trees
5. Transfer pool creation fee (100 GNS)
6. Save pool to pools map

Requirements:

- Both tokens must be registered in GRC20 registry
- Only one pool per token pair + fee combination
- Creator must hold 100 GNS

4.3 Position Mint Flow

Process for creating a new liquidity position.



Creation Process:

1. User calls `Mint` with pool, price range, desired token amounts
2. Validate tick spacing (matches fee tier interval)
3. Call Pool.Mint:
   - Calculate maximum liquidity from token amounts
   - Update tick bitmap
   - Update position state
   - Return actual required token amounts
4. Transfer token0, token1 from user

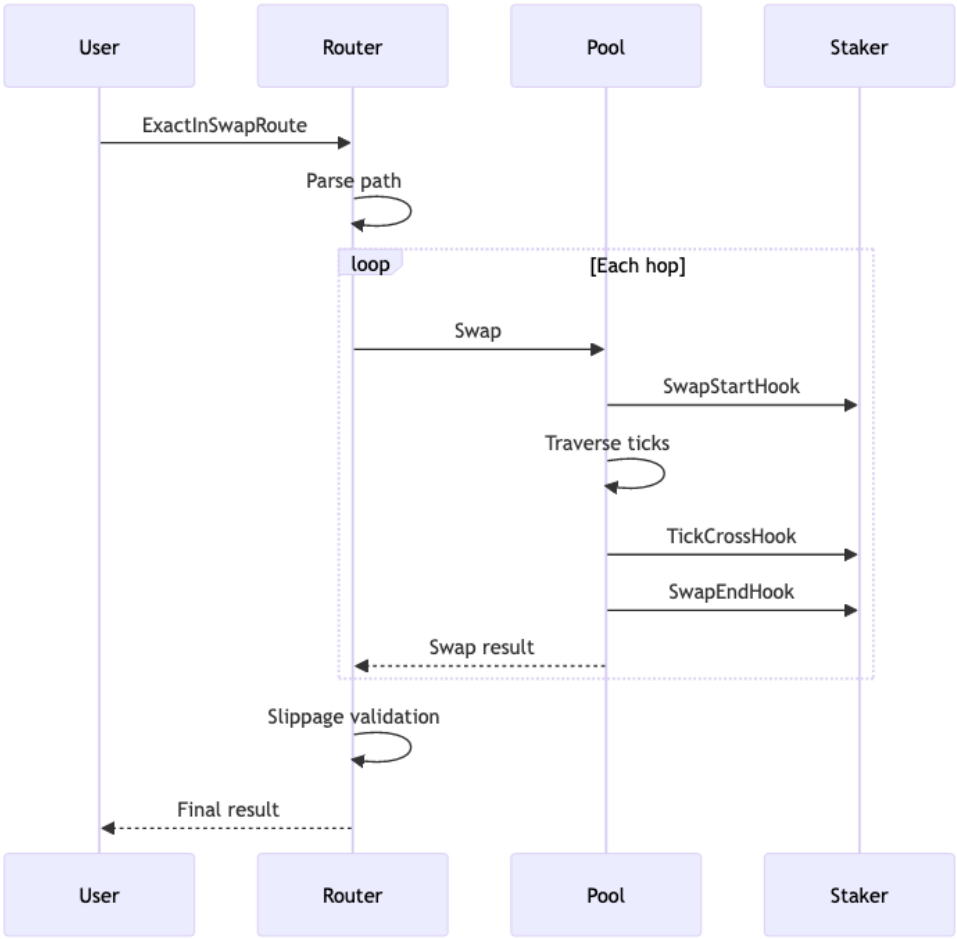
5. Mint GNFT NFT (dynamic SVG metadata)
6. Return tokenId, liquidity, actual token amounts used

Position Characteristics:

- Each position has a unique NFT ID
- Price range (tickLower, tickUpper) cannot be changed
- Liquidity add/remove handled by separate functions

4.4 Swap Execution Flow

Process for executing token swaps.



Swap Process:

1. User calls with input token, output token, path, minimum output amount
2. Router parses path string (multi-hop supported)
3. Execute Pool.Swap for each hop:
   - Acquire reentrancy lock
   - SwapStartHook → Notify Staker of batch start
   - Traverse ticks consuming liquidity, update price
   - Call TickCrossHook on tick boundary crossing
   - SwapEndHook → Notify Staker of batch end
   - Handle token transfer via callback

4. Verify final output amount meets minimum
5. Return result

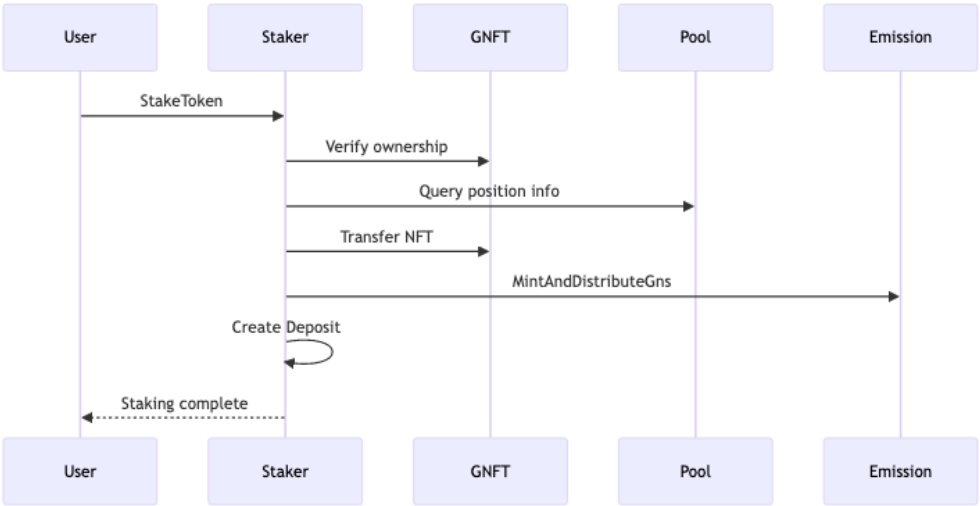
Hook System:

Notifies Staker of events during Pool swap to update reward calculations:

- `SwapStartHook`: Initialize batch processing
- `TickCrossHook`: Update accumulators on tick boundary crossing
- `SwapEndHook`: Complete batch processing

4.5 Staking Flow

Process for staking positions to receive rewards.



Staking Process:

1. User calls `StakeToken` with positionId
2. Verify ownership in GNFT
3. Query position info from Pool (liquidity, ticks)
4. Transfer NFT to Staker contract
5. Trigger Emission (GNS minting)
6. Create Deposit:
   - lpTokenId, targetPoolPath
   - liquidity, tick range
   - stakeTimestamp (for warm-up calculation)
   - Initialize reward accumulator
7. Emit staking complete event

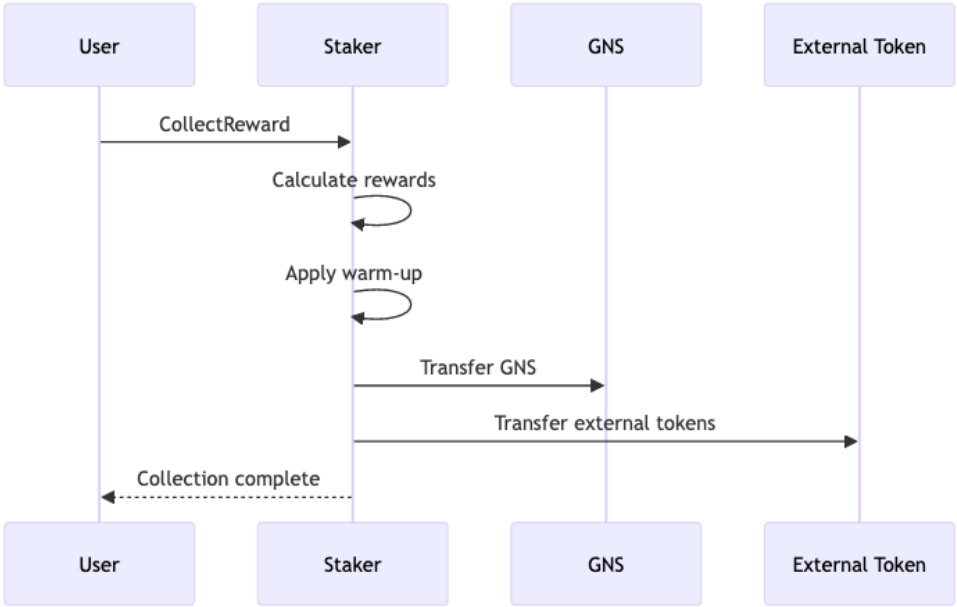
Deposit Data:

```
Deposit
├── lpTokenId         Position NFT ID
├── targetPoolPath    Staked pool
├── liquidity         Liquidity amount
├── tickLower/Upper   Price range
```

```
├── stakeTimestamp    Staking start time
└── rewardState       Reward accumulation state
```

4.6 Collect Reward Flow

Process for collecting staking rewards.



Reward Calculation:

1. Query global accumulated rewards
2. Calculate accumulated rewards within position's price range
3. Apply position liquidity ratio
4. Subtract previously collected amount
5. Apply warm-up ratio (based on staking duration)
6. Transfer GNS and external tokens
7. Update lastCollectionTime

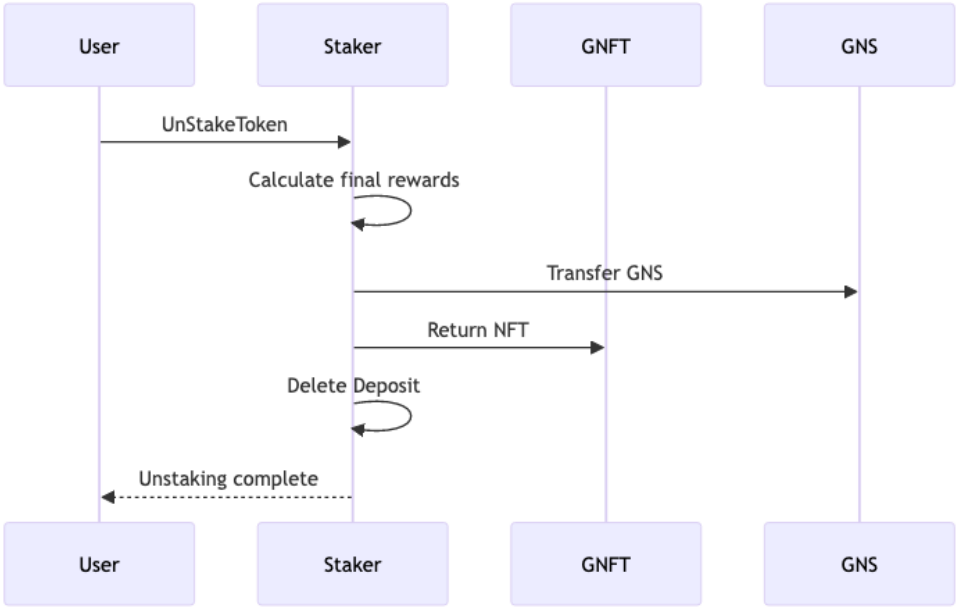
Warm-up Period:

To prevent gaming by new stakers, reward ratio increases with staking duration:

Duration	Reward Ratio
Day 0	30%
Days 1-7	50%
Days 7-14	70%
Day 14+	100%

4.7 Unstake Flow

Process for unstaking and reclaiming positions.



Unstaking Process:

1. Calculate and transfer final rewards (GNS + external tokens)
2. Return NFT to user
3. Delete Deposit data
4. Emit unstaking complete event

Subsequent User Actions:

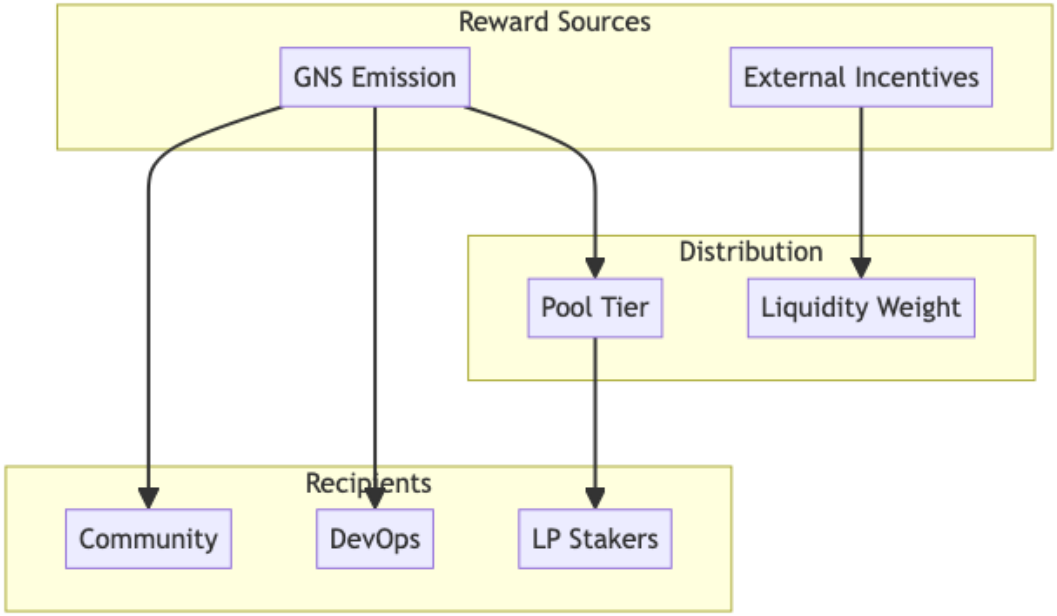
To reclaim liquidity after unstaking:

1. `Position.DecreaseLiquidity` - Remove liquidity
2. `Position.CollectFee` - Collect tokens and fees

5. Reward Distribution

5.1 Reward System Overview

GnoSwap's reward system provides rewards from two sources.



Internal Rewards (GNS):

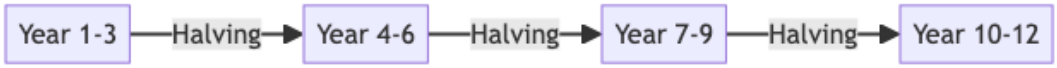
- Issued from Emission contract according to schedule
- Distributed to pools based on Pool Tier
- Distributed to stakers based on liquidity ratio

External Incentives:

- Anyone can set up custom token rewards for specific pools
- Evenly distributed over specified period
- Distributed to stakers based on liquidity ratio

5.2 GNS Emission Schedule

GNS is issued over 12 years with halving applied.



Category	Amount
Initial Mint	100 trillion GNS
Emission Pool	900 trillion GNS
Maximum Supply	1,000 trillion GNS
Duration	12 years
Halving	~Every 3 years

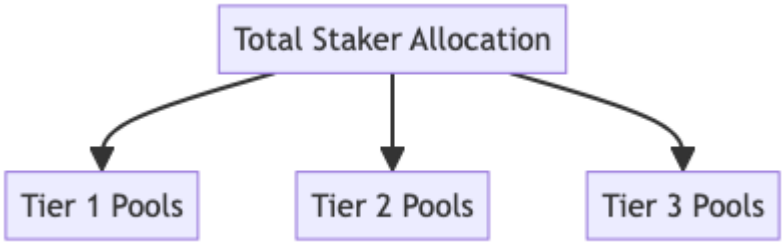
Emission Characteristics:

- Time-proportional issuance per block/transaction
- 50% reduction in per-second issuance at each halving

- Undistributed amounts carry over to next distribution

5.3 Pool Tier System

Pools receive differential GNS rewards based on tier.



Tier Structure:

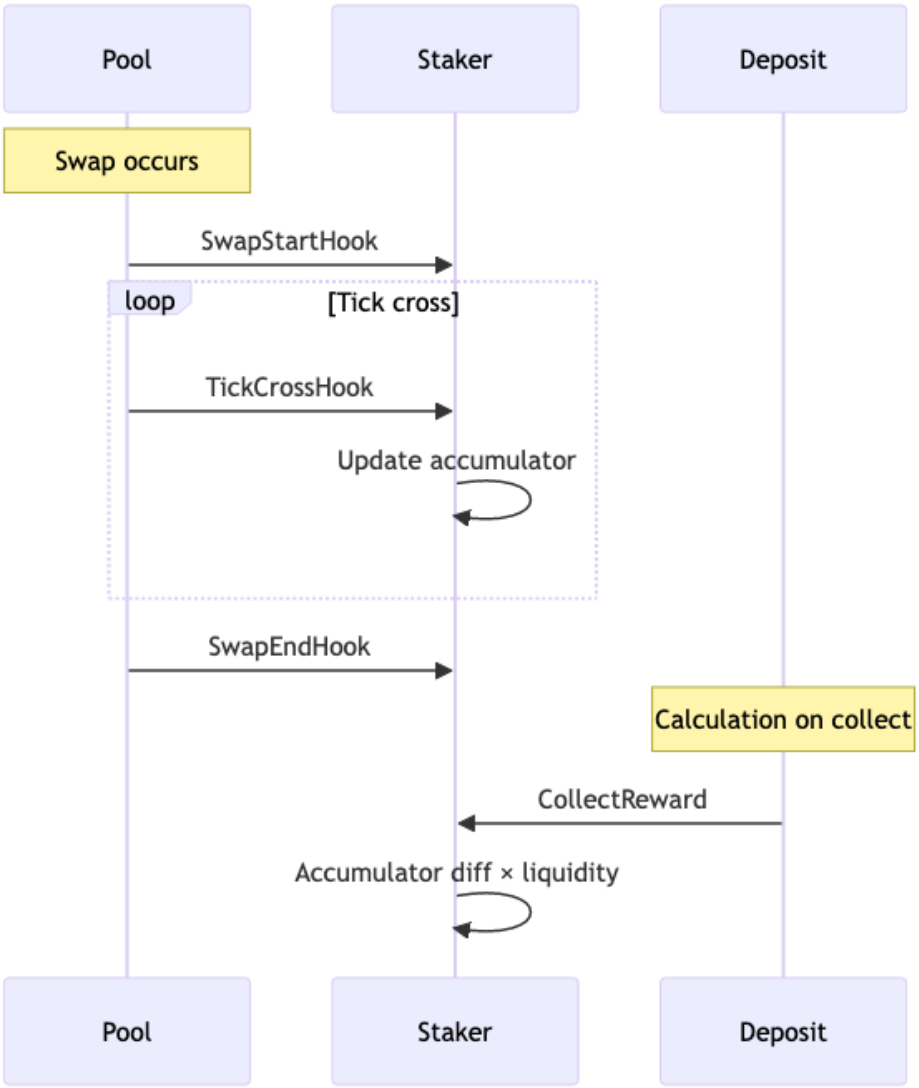
Tier	Description	Reward Weight
Tier 1	Core pairs	Highest
Tier 2	Major pairs	Medium
Tier 3	General pairs	Low

Distribution Logic:

1. Determine total Staker allocation (from Emission)
2. Allocate based on tier weights
3. Equal distribution among pools within each tier
4. Distribution to stakers within pool based on liquidity ratio

5.4 Reward Calculation

Reward accumulators are updated during swaps.



Calculation Formula:

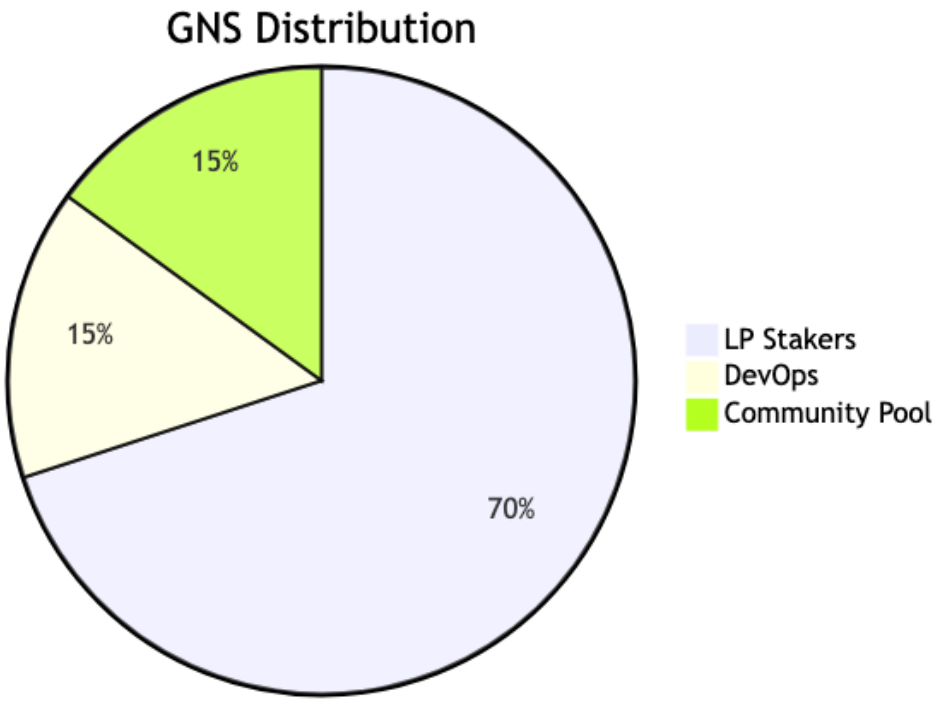
```
Individual Reward = (Current Accumulator - Previous Accumulator) ×
Liquidity × Warm-up Ratio
```

Accumulator Updates:

- `rewardPerLiquidityGlobal`: Reward per liquidity for entire pool
- `rewardPerLiquidityInside`: Reward within specific price range
- Inside value recalculated on tick boundary crossing

5.5 Distribution Targets

GNS Emission is distributed to multiple recipients.



Target	Description	Distribution Method
LP Stakers	Liquidity providers	Tier × Liquidity ratio
DevOps	Development/Operations team	Fixed ratio
Community Pool	Community treasury	Fixed ratio

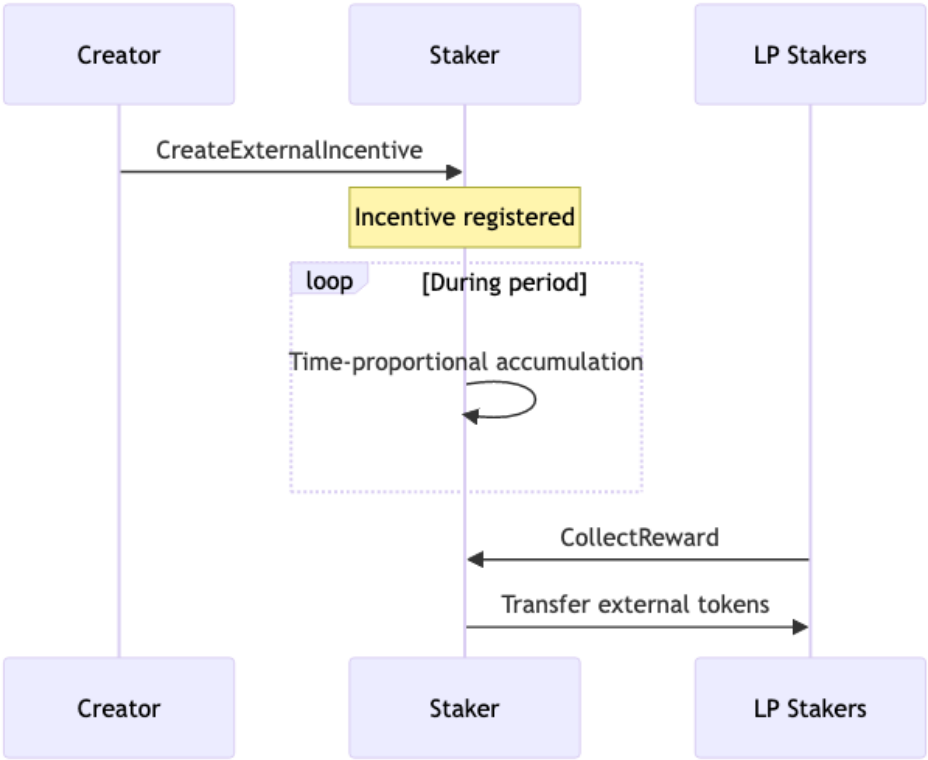
Protocol Fee Distribution:

A portion of swap fees is collected as protocol fees:

- 0-10% of swap fees (configurable)
- Distributed to DevOps and Community Pool

5.6 External Incentives

External projects can provide additional rewards to specific pools.



Incentive Settings:

Parameter	Description
targetPoolPath	Target pool
rewardToken	Reward token address
rewardAmount	Total reward amount
startTimestamp	Start time
endTimestamp	End time

Distribution Method:

- Even distribution per second during period
- Distribution to stakers based on liquidity ratio
- Minimum reward threshold can be applied

Appendix

A. Contract Addresses (Roles)

Role	Description
admin	System administrator
governance	Governance contract

Role	Description
pool	Pool domain
position	Position domain
router	Router domain
staker	Staker domain
emission	Emission contract
gns	GNS token
protocol_fee	Protocol fee

B. Fee Structure

Fee	Value	Description
Pool Creation	100 GNS	Pool creation cost
Swap Fee Tiers	0.01~1%	Per-pool swap fees
Protocol Fee	0−10%	Protocol share of swap fees
Router Fee	0.15%	Router fee

C. Key Constants

Constant	Value	Description
MIN_TICK	−887272	Minimum tick
MAX_TICK	887272	Maximum tick
Q96	2^96	Price precision
MAX_FEE	1000000	Maximum fee (100%)